



Assignment No.8

OBJECT ORIENTED PROGRAMMING

Karandeep Singh

1024150362

2034

FUNCTION TEMPLATE

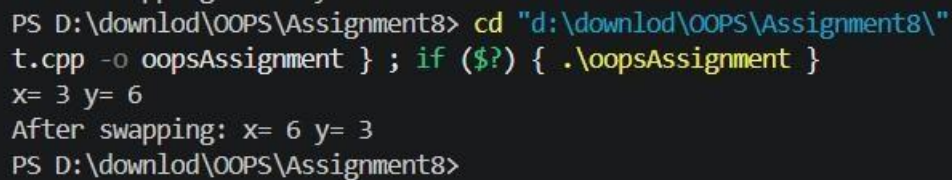
1. Write a function template to swap two variables of any data type.

CODE:

```
#include<iostream>
using namespace std;

template<class T>
void swapValue(T &a, T &b){
    T temp=a;   a=b;   b=temp;
}

int main(){
    int x=3;    int
    y=6;
    cout<<"x= "<<x<<" y= "<<y;   swapValue(x,y);
    cout<<"\nAfter swapping: "<<"x= "<<x<<" y= "<<y;
    return 0;
}
```



```
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\"
t.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
x= 3 y= 6
After swapping: x= 6 y= 3
PS D:\download\OOPS\Assignment8>
```

2. Write a function template to find the minimum element in an array.

CODE:

```
#include<iostream>
using namespace std;

template<class T>
T findMin(T arr[],int n){
    T min=arr[0];   for(int
    i=1;i<n;i++){
        if(arr[i]<min)
            min=arr[i];
    }   return
    min;
```

```
}
```

```
int main(){
    int
    arr[]={5,2,9,7,4,6}; cout<<"MIN:
    "<<findMin(arr,6); return 0;
}
```

```
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?) { g++ oopsAssignm
t.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
MIN: 2
PS D:\download\OOPS\Assignment8>
```

3. Write a function template to sort an array using bubble sort.

CODE:

```
#include<iostream>
using namespace std;
```

```
template<class T> void
bubbleSort(T arr[], int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(arr[j]>arr[j+1]){
                T
                temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}
```

```
int main(){
    int
    arr[]={8,6,3,9,1};
    for(int i=0;i<5;i++){
        cout<<arr[i]<<" ";
    }
    bubbleSort(arr,5);
    cout<<"\nAfter sorting: ";
    for(int i=0;i<5;i++){
        cout<<arr[i]<<" ";
    }
    return 0;
}
```

```
After sorting: 1 3 6 8 9
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?) { g++ tempCodeRunne
rFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
8 6 3 9 1
After sorting: 1 3 6 8 9
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?) { g++ tempCodeRunne
```

4. Write a function template to perform linear search.

CODE:

```
#include<iostream>
using namespace std;
```

```
template<class T> int linearSearch(T
arr[],int n, T key){    for(int
i=0;i<n;i++){        if(arr[i]==key)
return i;
    }
return -1;
}
```

```
int    main(){    int
arr[]={8,6,3,9,1};    int
pos=linearSearch(arr,5,9);
cout<<"Position: "<<pos;
return 0;
}
```



```
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Position: 3
PS D:\download\OOPS\Assignment8>
```

5. Design overloaded versions of a function template process() such that:

i)Accepts a single parameter ii)Accepts two parameters of the same type iii) Accepts two parameters of different types

CODE:

```
#include <iostream> using
namespace std;
```

```
// i) Single parameter template <class T>
void process(T a) {    cout << "Single
parameter: " << a << endl;
}
```

```
// ii) Two same type parameters template <class T>
void process(T a, T b) {    cout << "Same type: "
<< a << ", " << b << endl;
}
```

```
// iii) Two different type parameters template <class T,
class U> void process(T a, U b) {    cout << "Different
types: " << a << ", " << b << endl;
}
```

```
int main() {
process(5);
process(5, 10);
process(5, 3.5);
return 0;
}
```

```
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?)
{ g++ oopsAssignment.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
Single parameter: 5
Same type: 5, 10
Different types: 5, 3.5
PS D:\download\OOPS\Assignment8>
```

Class Template

1. Develop a class template for stack with push and pop operations.

Code:

```
#include<iostream> using
namespace std;
```

```
template<class T> class
Stack{
    T arr[100];
    int top;

    public:
    stack(){
        top=-1;
    }
    void push(T x){
        arr[++top]=x;
    }
    void pop(){    if(top>=0)
        cout<<"Popped: "<<arr[top--]<<endl;
    }
};
```

```

int main(){
Stack<int> s;
s.push(3);
    s.push(6);
    s.pop();
return 0;
}

```

```

PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?)
{ g++ ClassTemplate.cpp -o ClassTemplate } ; if ($?) { .\ClassTemplate }
Popped: 6
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?)

```

2. Develop a class template for queue with enqueue and dequeue operations.

ANS2)

CODE:

```

#include <iostream> using
namespace std;

```

```

template <class T>
class Queue { T
arr[100];      int
front, rear;

```

```

public: Queue() {
front = rear = -1;
}

```

```

    void enqueue(T x) {
arr[++rear] = x;
}

```

```

    void dequeue() {      if (front < rear)      cout
<< "Dequeued: " << arr[++front] << endl;
}
};

```

```

int main() {
Queue<int> q;
q.enqueue(10);
q.enqueue(20);
q.dequeue();
}

```

```
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?) { g++ oopsAssignment.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
Dequeued: 10
PS D:\download\OOPS\Assignment8>
```

3. Create a class template to store and display a pair of values.

ANS3) CODE:

```
#include <iostream>
using namespace std;
```

```
template <class T1, class T2>
class Pair {    T1 a;
               T2 b;
```

```
public:
```

```
    Pair(T1 x, T2 y) {
a = x;    b = y;
    }
```

```
    void display() {    cout << "Pair:
" << a << ", " << b;
    }
};
```

```
int main() {
    Pair<int, float> p(10, 5.5);
p.display();
}
```

```
PS D:\download\OOPS\Assignment8> cd "d:\download\OOPS\Assignment8\" ; if ($?) { g++ oopsAssignment.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
Pair: 10, 5.5
PS D:\download\OOPS\Assignment8>
```

4. Create a class template for basic arithmetic operations.

ANS4)

CODE:

```
#include <iostream>
using namespace std;
```

```
template <class T>
class Arithmetic {
    T a, b;
```

```
public:
```

```

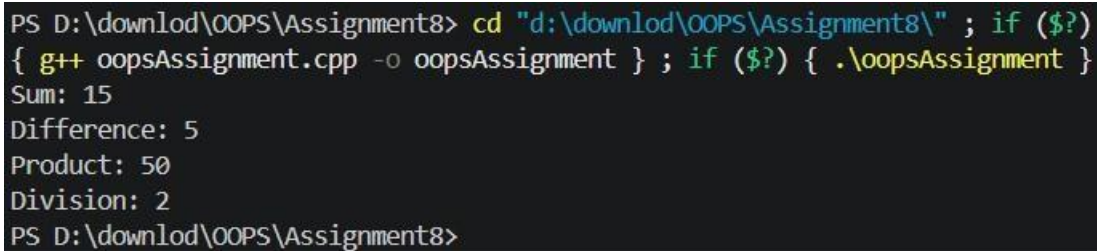
    Arithmetic(T x, T y) {
a = x;      b = y;
    }

    void add() { cout << "Sum: " << a + b << endl; }
void sub() { cout << "Difference: " << a - b << endl; }
void mul() { cout << "Product: " << a * b << endl; }
void div() { cout << "Division: " << a / b << endl; }
};

int main() {
    Arithmetic<int> obj(10, 5);
obj.add();   obj.sub();
obj.mul();   obj.div();

}

```



```

PS D:\downlod\OOPS\Assignment8> cd "d:\downlod\OOPS\Assignment8\" ; if ($?)
{ g++ oopsAssignment.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
Sum: 15
Difference: 5
Product: 50
Division: 2
PS D:\downlod\OOPS\Assignment8>

```

5. Create a class template to input and display array elements. ANS5)

CODE:

```

#include <iostream>
using namespace std;

template <class T>
class Array {    T
arr[100];    int n;

public:
    void input() {        cout
<< "Enter size: ";        cin
>> n;

        for (int i = 0; i < n; i++)
cin >> arr[i];
    }
}

```



```

        void display() {      for
(int i = 0; i < n; i++)
cout << arr[i] << " ";
    }
};

```

```

int  main()  {
Array<int>  a;
a.input();
    a.display();
}

```

```

PS D:\downlod\OOPS\Assignment8> cd "d:\downlod\OOPS\Assignment8\" ; if ($?)
{ g++ oopsAssignment.cpp -o oopsAssignment } ; if ($?) { .\oopsAssignment }
Enter size: 3
1
2
3
1 2 3
PS D:\downlod\OOPS\Assignment8>

```